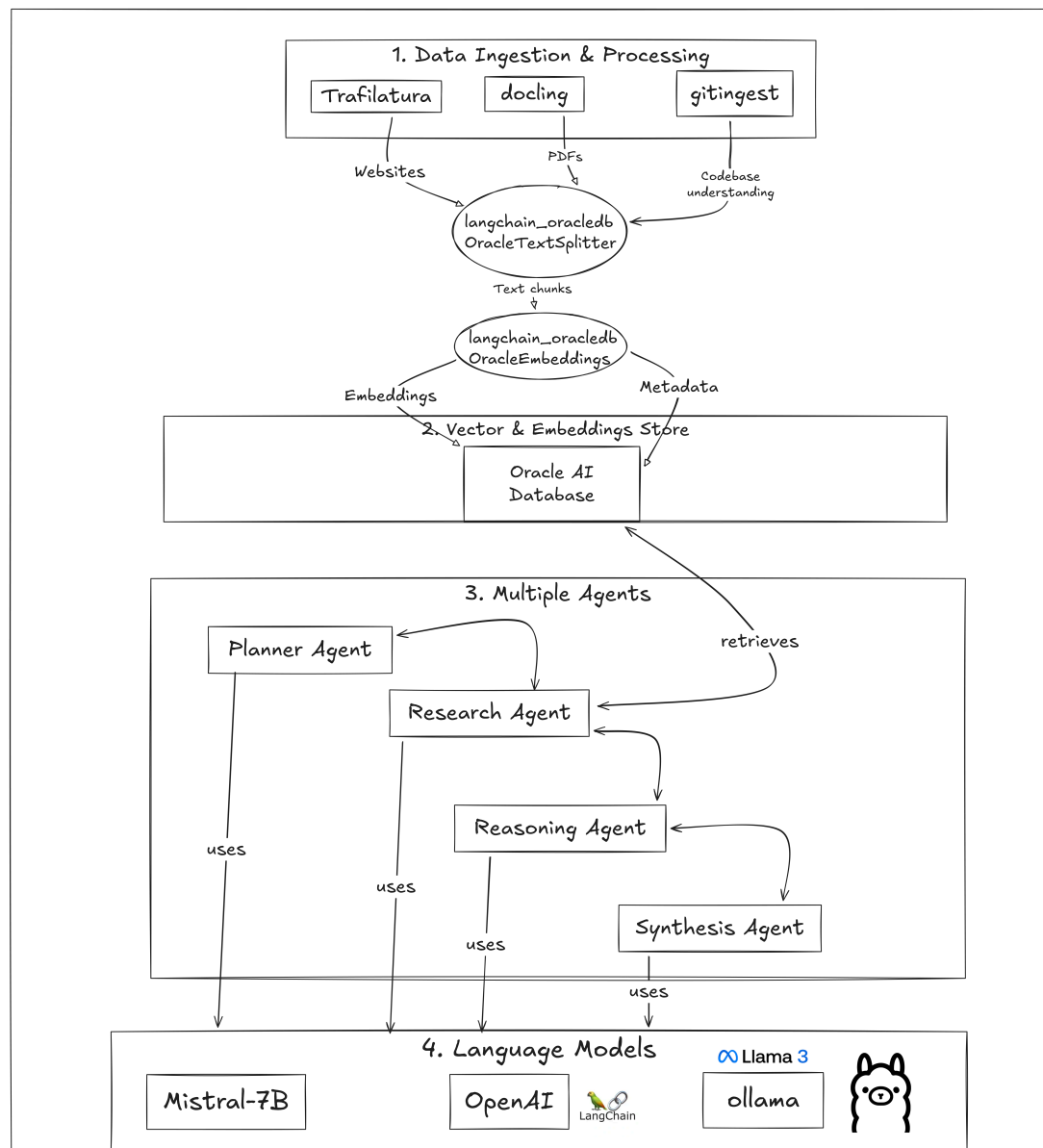


Agentic RAG System

Introduction

An intelligent RAG (Retrieval Augmented Generation) system that uses an LLM agent to make decisions about information retrieval and response generation. The system processes PDF documents and can intelligently decide which knowledge base to query based on the user's question.

OKE / OCI GPU Compute



The system has the following features:

- Intelligent query routing
- PDF processing using Docling for accurate text extraction and chunking
- Persistent vector storage with Oracle AI Database (PDF and Websites)
- Smart context retrieval and response generation
- FastAPI-based REST API for document upload and querying
- Support for local, agentic workflows using **gemma3:270m** via Ollama

- A2A Agent RAC System

Upload PDFs, process web content, repositories, and chat with your documents using local or OpenAI models.

Note on Performance: When using the Local (Mistral) model, initial loading can take 1-5 minutes, and each query may take 30-60 seconds to process depending on your hardware. OpenAI queries are typically much faster.

🟢

A2A Protocol v1.0 is active and being used for vector storage.

Model Management

Model Selection

Choose your preferred model for the conversation.

Select Model

Choose the model to use for the conversation

qwq

Download Selected Model

Download Status

Select a model and click Download to begin...

Model FAQ

Model	Parameters	Size	Download Command
qwq	32B	20GB	qwq:latest
gemma3	4B	3.3GB	gemma3:latest
llama3.3	70B	43GB	llama3.3:latest
phi4	14B	9.1GB	phi4:latest
mistral	7B	4.1GB	mistral:latest

Model ManagementDocument ProcessingChatA2A Protocol TestingA2A Demo

A2A Agent Swapping Demo

This demo showcases the Agent-to-Agent (A2A) Protocol's ability to dynamically swap agents based on availability. Toggle agent availability and run tasks to see the orchestrator automatically select the best available agent.

1. Agent Availability

Toggle which agents are able to accept tasks.

Planner Agents
 - ☒ Planner Agent A (v1.0)
 - ☒ Planner Agent B (v1 - Fast)Reasoner Agents
 - ☒ Researcher Agent A (Web)
 - ☒ Researcher Agent B (PDF/Vector)Reasoner Agents
 - ☒ Reasoner Agent A (DeepThink)
 - ☒ Reasoner Agent B (QuickLogic)Synthesizer Agents
 - ☒ Synthesizer Agent A (Creative)
 - ☒ Synthesizer Agent B (Concise)

2. Task Simulation

Start Multi-Agent TaskSimulate Researcher Swap

Task Trace

Starting new A2A demo simulation tasks...
Target Query: What are Generative Adversarial Networks (GANs)?
Task ID: a2a-demo-1768592613-7885

Orchestrator: Discovering Planner Agents...

Selected: Planner A (v1.0)

Planner A (v1.0): Decomposing task into sub-steps...

1. Define GANs

2. Explain architecture (Generator/Discriminator)

3. List applications

Orchestrator: Discovering Researcher Agents...

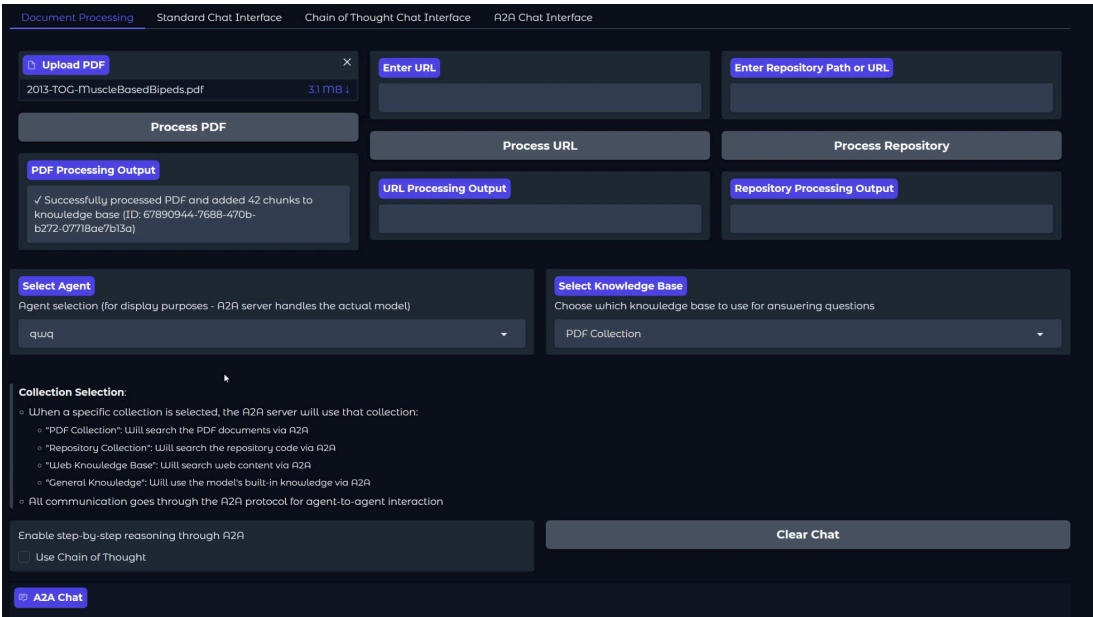
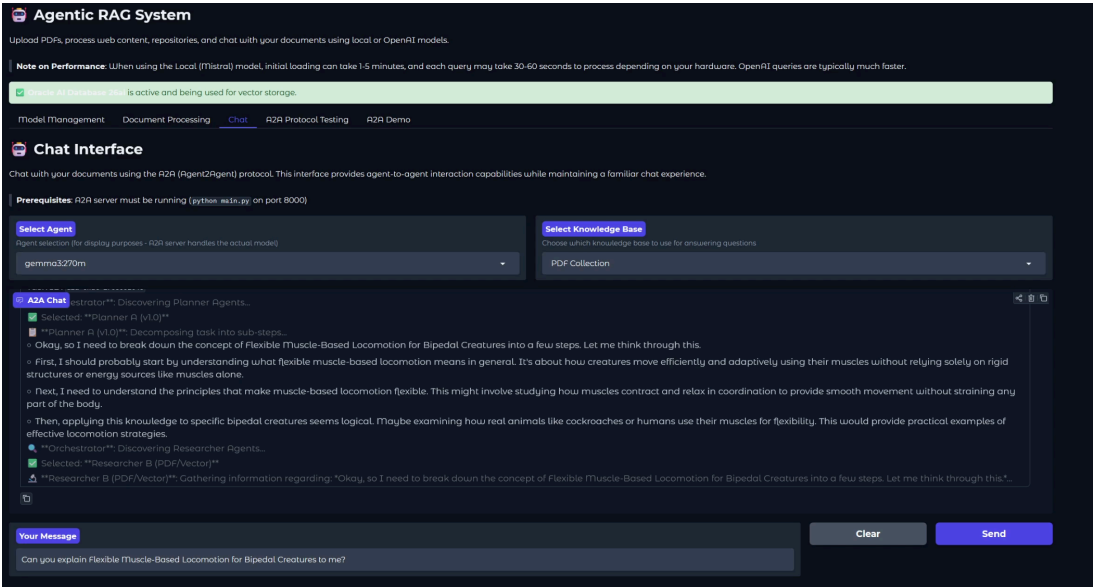
Selected: Researcher A (Web)

Researcher A (Web): Gathering information...

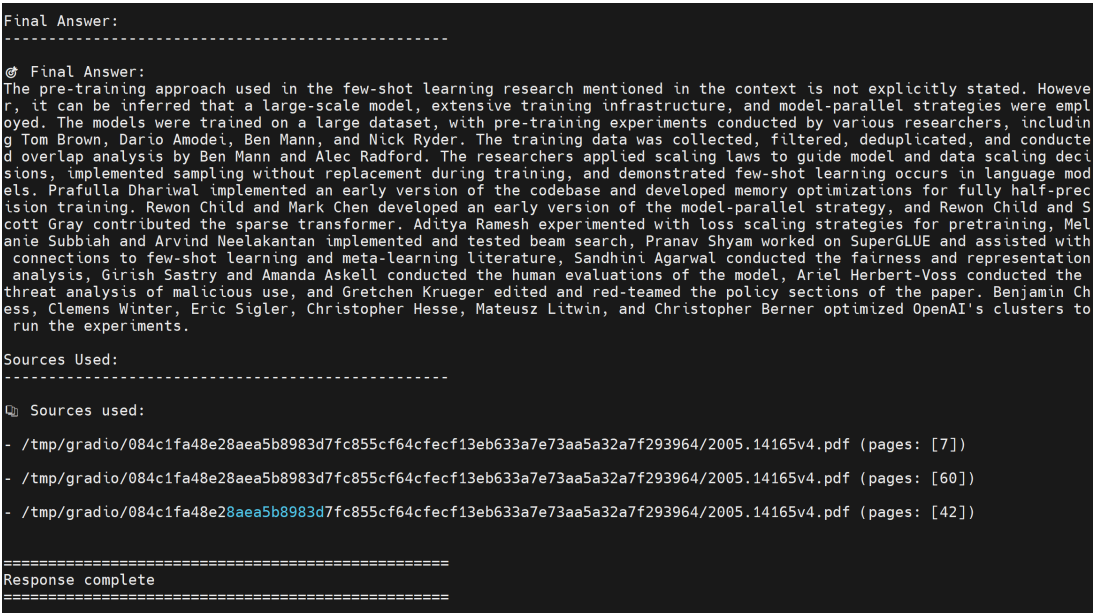
View Agent Cards

Registered Agent Cards

PlannersResearchersReasonersSynthesizers



Here you can find a result of using Chain of Thought (CoT) reasoning:



0. Prerequisites and setup

Prerequisites

1. Install and run **Ollama** (required for local LLM inference):

- [Download Ollama](#)
- Pull the default model:

```
ollama pull gemma3:270m
# or the 4b model
ollama pull gemma3:latest
```

2. (Optional) For specialized agents, you may pull other Ollama models:

```
# for coding
ollama pull qwen2.5-coder:7b
ollama pull deepseek-r1:1.5b
```

Setup

1. Clone the repository and install dependencies:

```
git clone https://github.com/oracle-devrel/ai-solutions.git
cd ai-solutions/apps/agentik_rag
pip install -r requirements.txt
```

2. Start the Ollama service

3. Pull the models you want to use beforehand:

```
ollama pull <model> # see ollama.com/models
```

1. Getting Started

You can launch this solution in multiple ways:

Quick Start: Unified Launcher (Recommended)

The simplest way to start the entire system is using the unified launcher:

```
# Start everything (FastAPI + Gradio + Open WebUI)
python run_app.py

# Start only Gradio interface
python run_app.py --gradio
```

```
# Start only Open WebUI interface
python run_app.py --openwebui

# Start API server only (for external frontends)
python run_app.py --api-only
```

Default Ports:

Service	Port	URL
FastAPI Backend	8000	http://localhost:8000
Gradio UI	7860	http://localhost:7860
Open WebUI	3000	http://localhost:3000

1. Using the Complete REST API

Start the API server:

```
# Option 1: Start the CLI (Command Line Interface)
python agent_cli.py

# Option 2: Start the GUI (Gradio Interface)
python gradio_app.py

# Alternative: Start the API server directly
python -m src.main
```

The API will be available at <http://localhost:8000>. You can then use the API endpoints as described in the API Endpoints section below.

2. Using the Gradio Interface (Recommended)

The system provides a user-friendly web interface using Gradio, which allows you to:

- Select and pull **ollama** models directly from the interface
- Upload and process PDF documents
- Process web content from URLs
- Chat with your documents using either local or OpenAI models
- Toggle Chain of Thought reasoning

To launch the Gradio interface:

```
python gradio_app.py
```

This will start the Gradio server and automatically:

1. Start the A2A API server in the background (required for agent collaboration)
2. Open the interface in your default browser at <http://localhost:7860>

The interface has two main tabs:

1. **Model Management:**

- Download models in advance to prepare them for use
- View model information including size and VRAM requirements
- Check download status and error messages

2. **Document Processing:**

- Upload PDFs using the file uploader
- Process web content by entering URLs
- View processing status and results

3. **Chat Interface:**

- Select between different model options:
 - **gemma3:270m** - Default local model (recommended)
 - Other Ollama models (if installed)
- Toggle Chain of Thought reasoning for more detailed responses
- Chat with your documents using natural language
- Clear chat history as needed

Note: The interface will automatically detect available models based on your configuration:

- Ollama models require Ollama to be installed and running

3. Using Open WebUI Interface

Open WebUI provides a modern, feature-rich chat interface that connects to the same backend as Gradio. It's ideal for users who prefer a ChatGPT-like experience.

Installation

```
pip install open-webui
```

Starting Open WebUI

```
# Option 1: Using the unified launcher (recommended)
python run_app.py --openwebui

# Option 2: Start Open WebUI standalone
python openwebui_app.py
```

```
# Option 3: Start both Gradio and Open WebUI
python run_app.py
```

Open WebUI will be available at <http://localhost:3000>.

Available Reasoning Models

Open WebUI sees 18 reasoning "models" that correspond to different reasoning strategies:

Model	Description
<code>standard</code>	Standard response without specialized reasoning
<code>standard-rag</code>	Standard response with RAG context from all collections
<code>cot</code>	Chain of Thought - step-by-step reasoning
<code>cot-rag</code>	Chain of Thought with RAG context
<code>tot</code>	Tree of Thoughts - parallel path exploration
<code>tot-rag</code>	Tree of Thoughts with RAG context
<code>react</code>	ReAct - reasoning and acting interleaved
<code>react-rag</code>	ReAct with RAG context
<code>self-reflection</code>	Self-Reflection - iterative critique and refinement
<code>self-reflection-rag</code>	Self-Reflection with RAG context
<code>consistency</code>	Self-Consistency - multiple samples with voting
<code>consistency-rag</code>	Self-Consistency with RAG context
<code>decomposed</code>	Decomposed - breaks problems into sub-problems
<code>decomposed-rag</code>	Decomposed with RAG context
<code>least-to-most</code>	Least-to-Most - simplest to most complex
<code>least-to-most-rag</code>	Least-to-Most with RAG context
<code>recursive</code>	Recursive - recursive problem decomposition
<code>recursive-rag</code>	Recursive with RAG context

Note: Models with `-rag` suffix perform unified similarity search across **all collections** (PDF, Web, Repository) before reasoning.

OpenAI-Compatible API

The backend exposes OpenAI-compatible endpoints that Open WebUI (and other clients) can consume:

```
# List available models
curl http://localhost:8000/v1/models

# Chat completion (streaming)
curl -X POST http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "cot-rag",
    "messages": [{"role": "user", "content": "What is machine learning?"}],
    "stream": true
  }'

# Health check
curl http://localhost:8000/v1/health
```

4. Using the CLI

The easiest way to use the system is through the interactive CLI:

```
python agent_cli.py
```

Interactive Experience:

```
AGENTIC RAG SYSTEM CLI
Oracle AI Vector Search + Ollama (Gemma 3)
```

```
? Select a Task:
Process PDFs
Process Websites
Manage Vector Store
Test Oracle DB
Chat with Agent (RAG)
Exit
```

Features:

1. **Process PDFs:** Ingest documents from files, directories, or URLs.
2. **Process Websites:** Crawl and ingest web content.
3. **Manage Vector Store:** Add processed chunks to the database.
4. **Test Oracle DB:** Verify connectivity and view table statistics.
5. **Chat with Agent:** Interactive RAG chat using `gemma3:270m` (Ollama).

5. Component-level Usage (Advanced)

If you prefer to run individual components manually:

Process PDFs

To process a PDF file and save the chunks to a JSON file, run:

```
# Process a single PDF
python -m src.pdf_processor --input path/to/document.pdf --output chunks.json

# Process multiple PDFs in a directory
python -m src.pdf_processor --input path/to/pdf/directory --output chunks.json

# Process a single PDF from a URL
python -m src.pdf_processor --input https://example.com/document.pdf --output chunks.json
# sample pdf: https://arxiv.org/pdf/2203.06605
```

Process Websites with Trafilatura

Process a single website and save the content to a JSON file:

```
python -m src.web_processor --input https://example.com --output docs/web_content.json
```

Or, process multiple URLs from a file and save them into a single JSON file:

```
python -m src.web_processor --input urls.txt --output docs/web_content.json
```

Manage Vector Store

To add documents to the vector store and query them, run:

```
# Add documents from a chunks file, by default to the pdf_collection
python -m src.store --add chunks.json
# for websites, use the --add-web flag
python -m src.store --add-web docs/web_content.json

# Query the vector store directly, both pdf and web collections
# llm will make the best decision on which collection to query based upon your input
python -m src.store --query "your search query"
python -m src.local_rag_agent --query "your search query"
```

Test Oracle DB Vector Store

The system includes a test script to verify Oracle DB connectivity and examine the contents of your collections. This is useful for:

- Checking if Oracle DB is properly configured
- Viewing statistics about your collections
- Inspecting the content stored in each collection
- Testing basic vector search functionality

To run the test:

```
# Basic test - checks connection and runs a test query
python tests/test_oradb.py

# Show only collection statistics without inserting test data
python tests/test_oradb.py --stats-only

# Specify a custom query for testing
python tests/test_oradb.py --query "artificial intelligence"
```

The script will:

1. Verify Oracle DB credentials in your `config.yaml` file
2. Test connection to the Oracle DB
3. Display the total number of chunks in each collection (PDF, Web, Repository, General Knowledge)
4. Show content and metadata from the most recently inserted chunk in each collection
5. Unless running with `--stats-only`, insert test data and run a sample vector search

Requirements:

- Oracle DB credentials properly configured in `config.yaml`:

```
ORACLE_DB_USERNAME: ADMIN
ORACLE_DB_PASSWORD: your_password_here
ORACLE_DB_DSN: your_connection_string_here
```

- The `oracledb` Python package installed

Use RAG Agent

To query documents using the local Ollama model, run:

```
# Using local ollama model (gemma3:270m by default)
python -m src.local_rag_agent --query "Can you explain the DaGAN Approach proposed
in the Depth-Aware Generative Adversarial Network for Talking Head Video
Generation article?"
```

6. Complete Pipeline Example

First, we process a document and query it using the local model. Then, we add the document to the vector store and query from the knowledge base to get the RAG system in action.

```
# 1. Process the PDF
python -m src.pdf_processor --input example.pdf --output chunks.json

#python -m src.pdf_processor --input https://arxiv.org/pdf/2203.06605 --output
chunks.json

# 2. Add to vector store
python -m src.store --add chunks.json

# 3. Query using local model
python -m src.local_rag_agent --query "Can you explain the DaGAN Approach proposed
in the Depth-Aware Generative Adversarial Network for Talking Head Video
Generation article?"
```

2. Deployment

Docker Deployment

You can deploy the application using Docker. This ensures a consistent environment with all dependencies pre-installed.

1. Build the Docker image:

```
docker build --network=host -t agentic-rag .
```

2. Run the container:

```
# Recommended for Linux (bypasses Docker network/DNS issues)
docker run -d \
  --network=host \
  --gpus all \
  --name agentic-rag \
  agentic-rag

# Alternative (Port mapping)
# Note: May require DNS configuration if container cannot access
host/internet
# docker run -d \
#   --gpus all \
#   -p 7860:7860 \
#   -p 11434:11434 \
```

```
# --name agentic-rag \  
# agentic-rag
```

Note: The `--gpus all` flag requires the NVIDIA Container Toolkit. If you don't have a GPU, the application will run in CPU-only mode (slower), and you can omit this flag.

3. Access the application:

- Gradio Interface: <http://localhost:7860>
- Ollama API: <http://localhost:11434>

Kubernetes Deployment

For Kubernetes deployment, we provide a comprehensive set of manifests and scripts in the `k8s/` directory.

1. Prerequisites:

- A Kubernetes cluster (local or cloud)
- `kubectl` configured
- NVIDIA GPU nodes (recommended for best performance)

2. Deploy using the helper script:

```
cd k8s  
  
# Deploy to default namespace  
./deploy.sh  
  
# Or deploy with a specific Hugging Face token (if needed)  
./deploy.sh --hf-token "your-token"
```

3. Manual Deployment:

```
kubectl apply -f k8s/local-deployment/
```

For detailed Kubernetes instructions, including OKE (Oracle Kubernetes Engine) and Minikube, please refer to the [Kubernetes README](#).

3. Chain of Thought (CoT) Support

The system implements an advanced multi-agent Chain of Thought system, allowing complex queries to be broken down and processed through multiple specialized agents. This feature enhances the reasoning capabilities of both local and cloud-based models.

Multi-Agent System

The CoT system consists of four specialized agents:

1. **Planner Agent:** Breaks down complex queries into clear, manageable steps
2. **Research Agent:** Gathers and analyzes relevant information from knowledge bases
3. **Reasoning Agent:** Applies logical analysis to information and draws conclusions
4. **Synthesis Agent:** Combines multiple pieces of information into a coherent response

Using CoT

You can activate the multi-agent CoT system in several ways:

1. Command Line:

```
# Using local gemma3:270m model (default)
python local_rag_agent.py --query "your query" --use-cot
```

2. Testing the System:

```
# Test with local model (default)
python tests/test_new_cot.py
```

3. API Endpoint:

```
POST /query
Content-Type: application/json

{
  "query": "your query",
  "use_cot": true
}
```

Example Output

When CoT is enabled, the system will show:

- The initial plan for answering the query
- Research findings for each step
- Reasoning process and conclusions
- Final synthesized answer
- Sources used from the knowledge base

Example:

```
Step 1: Planning
- Break down the technical components
- Identify key features
- Analyze implementation details
```

```
Step 2: Research
[Research findings for each step...]

Step 3: Reasoning
[Logical analysis and conclusions...]

Final Answer:
[Comprehensive response synthesized from all steps...]

Sources used:
- document.pdf (pages: 1, 2, 3)
- implementation.py
```

Benefits

The multi-agent CoT approach offers several advantages:

- More structured and thorough analysis of complex queries
- Better integration with knowledge bases
- Transparent reasoning process
- Improved answer quality through specialized agents
- Works with both local and cloud-based models

Annex: API Endpoints

Upload PDF

```
POST /upload/pdf
Content-Type: multipart/form-data

file: <pdf-file>
```

This endpoint uploads and processes a PDF file, storing its contents in the vector database.

Query

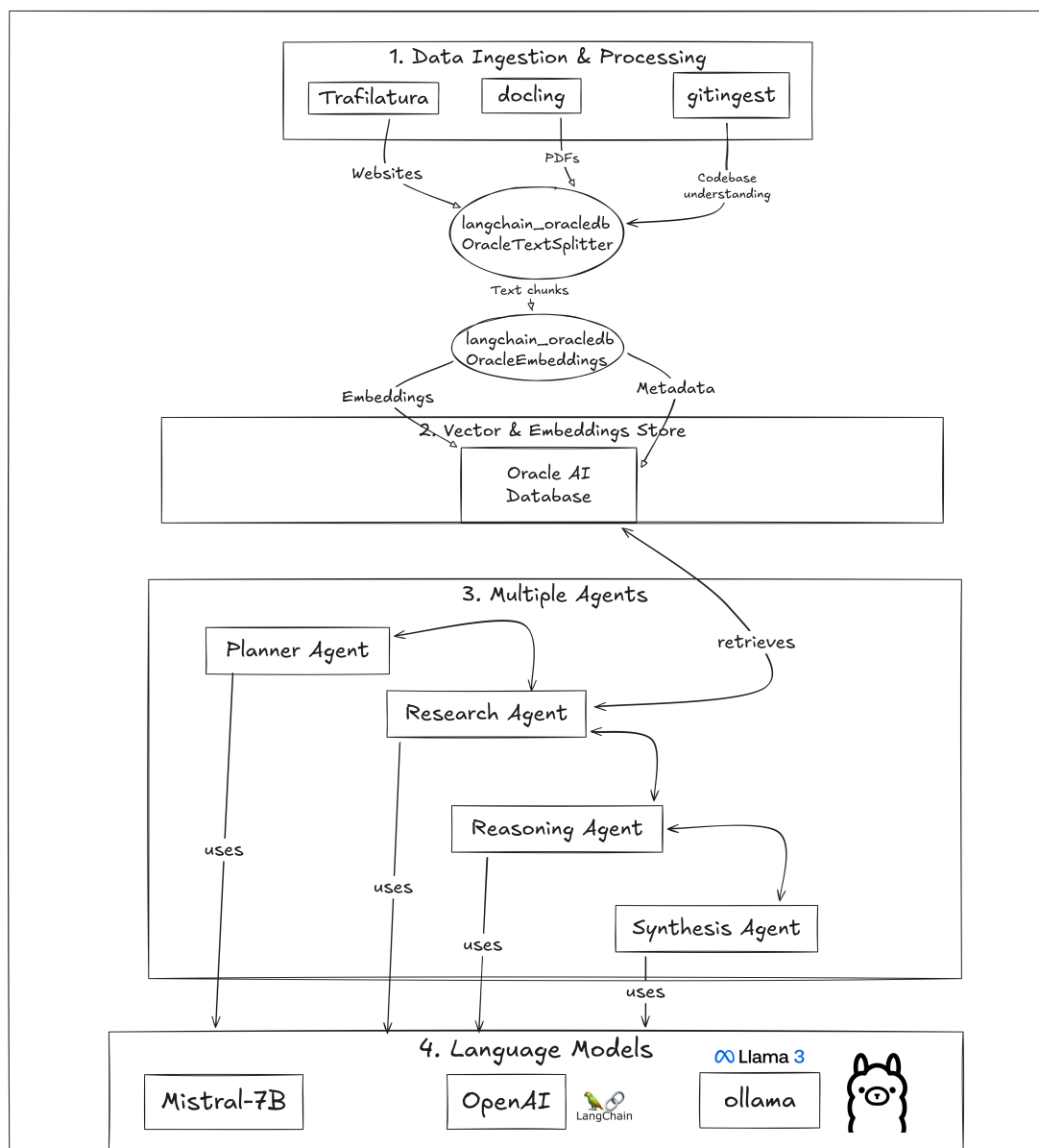
```
POST /query
Content-Type: application/json

{
  "query": "your question here"
}
```

This endpoint processes a query through the agentic RAG pipeline and returns a response with context.

Annex: Architecture

OKE / OCI GPU Compute



The system consists of several key components:

- 1. PDF Processor:** we use *docling* to extract and chunk text from PDF documents
- 2. Web Processor:** we use *trafilatura* to extract and chunk text from websites
- 3. GitHub Repository Processor:** we use *gitingest* to extract and chunk text from repositories
- 4. Vector Store:** Manages document embeddings and similarity search using *Oracle AI Database* (default) or *ChromaDB* (fallback)
- 5. RAG Agent:** Makes intelligent decisions about query routing and response generation
 - Uses *gemma3:270m* via Ollama as the default local model
- 6. FastAPI Server:** Provides REST API endpoints for document upload and querying
- 7. Gradio Interface:** Provides a user-friendly web interface for interacting with the RAG system

The RAG Agent flow is the following:

1. Analyzes query type
2. Try to find relevant PDF context, regardless of query type
3. If PDF context is found, use it to generate a response.
4. If no PDF context is found OR if it's a general knowledge query, use the pre-trained LLM directly

5. Fall back to a "no information" response only in edge cases.

Annex: Command Line Usage

You can run the system from the command line using:

```
python -m src.local_rag_agent --query "Your question here" [options]
```

Command Line Arguments

Argument	Description	Default
<code>--query</code>	The query to process	<i>Required</i>
<code>--embeddings</code>	Select embeddings backend (<code>oracle</code> or <code>chromadb</code>)	<code>oracle</code>
<code>--model</code>	Model to use for inference	<code>gemma3:270m</code>
<code>--collection</code>	Collection to query (PDF, Repository, Web, General)	Auto-determined
<code>--use-cot</code>	Enable Chain of Thought reasoning	<code>False</code>
<code>--store-path</code>	Path to ChromaDB store (if using ChromaDB)	<code>embeddings</code>
<code>--skip-analysis</code>	Skip query analysis step	<code>False</code>
<code>--verbose</code>	Show full content of sources	<code>False</code>
<code>--quiet</code>	Disable verbose logging	<code>False</code>

Examples

Query using Oracle DB (default):

```
python -m src.local_rag_agent --query "How does vector search work?"
```

Force using ChromaDB:

```
python -m src.local_rag_agent --query "How does vector search work?" --embeddings chromadb
```

Query with Chain of Thought reasoning:

```
python -m src.local_rag_agent --query "Explain the difference between RAG and fine-tuning" --use-cot
```

Query a specific collection:

```
python -m src.local_rag_agent --query "How to implement a queue?" --collection "Repository Collection"
```

Annex: User Interface Instructions

Unified Launcher Options

```
python run_app.py           # Start all UIs (default)
python run_app.py --gradio   # Gradio only (port 7860)
python run_app.py --openwebui # Open WebUI only (port 3000)
python run_app.py --api-only  # Backend API only (port 8000)
```

Open WebUI Features

Open WebUI provides a modern chat interface with:

- **18 Reasoning Models:** Each reasoning strategy appears as a selectable "model"
- **RAG Integration:** Models ending in **-rag** search all collections automatically
- **Streaming Responses:** Real-time streaming for better UX
- **Chat History:** Built-in conversation management
- **Export/Import:** Save and load conversations
- **Themes:** Dark/light mode support

Gradio Interface Instructions

1. Document Processing

- **Upload PDFs** using the file uploader.
- **Process web content** by entering URLs.
- **Process repositories** by entering paths or GitHub URLs.
- All processed content is added to the knowledge base.

2. Standard Chat Interface

- Quick responses without detailed reasoning steps.
- Select your preferred agent (Ollama gemma3 by default).
- Select which knowledge collection to query:
 - **PDF Collection:** Always searches PDF documents.
 - **Repository Collection:** Always searches code repositories.
 - **Web Knowledge Base:** Always searches web content.
 - **General Knowledge:** Uses the model's built-in knowledge without searching collections.

3. Chain of Thought Chat Interface

- Detailed responses with step-by-step reasoning.
- See the planning, research, reasoning, and synthesis steps.
- Great for complex queries or when you want to understand the reasoning process.
- May take longer but provides more detailed and thorough answers.
- Same collection selection options as the Standard Chat Interface.

4. A2A Chat Interface

- Same chat experience as standard interfaces but uses A2A protocol.
- **Granular Execution Trace:** Displays step-by-step execution including Orchestrator logic, Agent Selection, and detailed intermediate steps.
- **Real Vector Retrieval:** Shows actual retrieved content from the knowledge base during the Research phase, not just final answers.
- **Agent-to-Agent Communication:** All queries go through A2A protocol.
- **Collection Support:** PDF, Repository, Web, and General Knowledge collections.
- **Chain of Thought:** Step-by-step reasoning through A2A.
- **Status Monitoring:** Check A2A server connectivity.
- **Same UI:** Familiar chat interface with A2A backend.

5. A2A Protocol Testing

- Test the Agent2Agent (A2A) protocol functionality.
- **Health Check:** Verify A2A server connectivity.
- **Agent Card:** Get agent capability information.
- **Agent Discovery:** Find agents with specific capabilities.
- **Document Query:** Test A2A document querying with different collections.
- **Task Management:** Create, monitor, and track long-running tasks.
- **Task Dashboard:** View all tracked tasks and their statuses.
- **Complete Test Suite:** Run all A2A tests in sequence.

6. Performance Expectations

- **Default Model:** gemma3:270m (Ollama) - Optimized for speed and quality.
- **Other Ollama models:** Supported if installed via `ollama pull`.
- **A2A requests:** Depends on A2A server performance and network latency.

Note: The interface will automatically detect available models based on your configuration:

- `gemma3:270m` is the default option (requires Ollama to be installed and running).

- Other Ollama models can be selected if available.
- A2A testing requires the A2A server to be running separately.

5. Agent2Agent (A2A) Protocol Integration

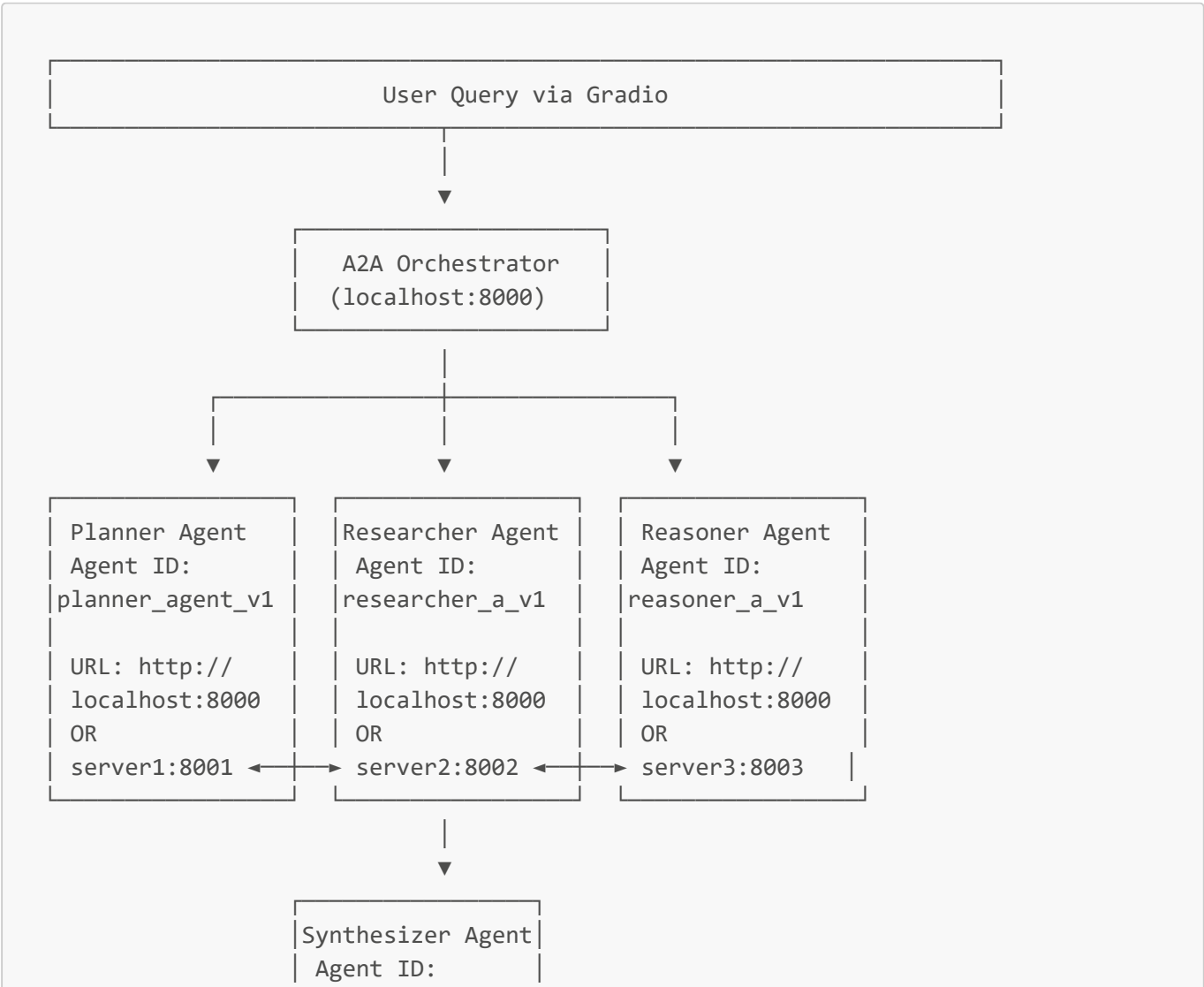
The agentic_rag system now includes full support for the Agent2Agent (A2A) protocol, enabling seamless communication and collaboration with other AI agents. This integration transforms the system into an interoperable agent that can participate in multi-agent workflows and ecosystems.

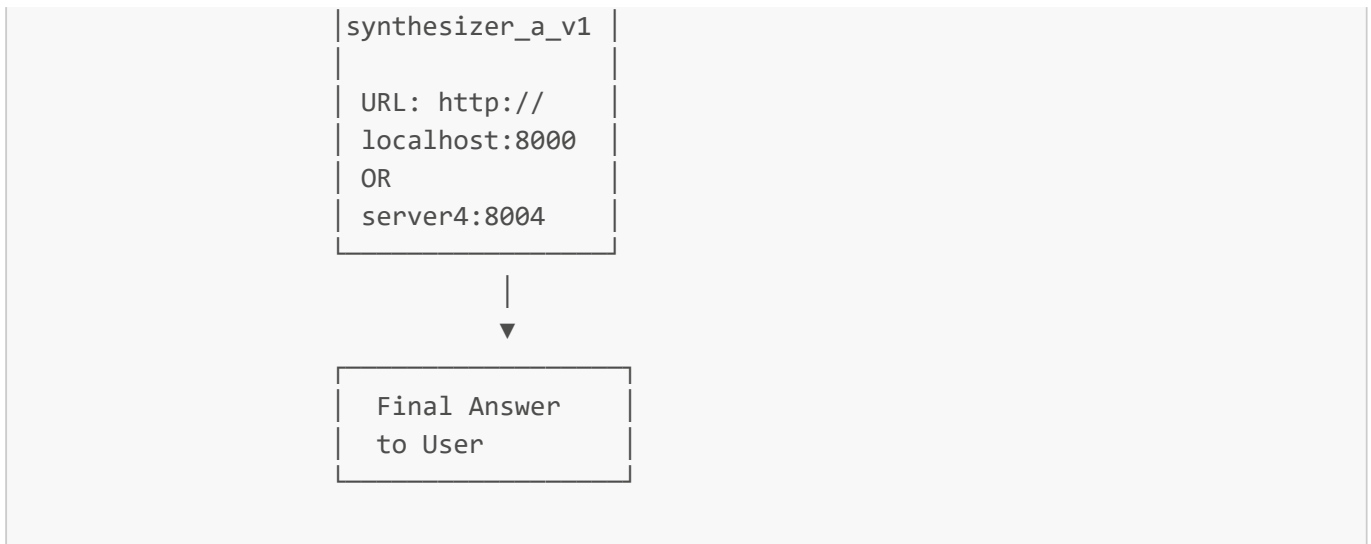
5.0 Distributed Chain of Thought Architecture

The system implements a **distributed multi-agent Chain of Thought (CoT)** architecture where each specialized agent can run on separate servers and communicate via the A2A protocol. This enables:

- **True Distributed Processing:** Each agent (Planner, Researcher, Reasoner, Synthesizer) can run on different servers
- **Scalable Agent Deployment:** Deploy agents independently based on resource requirements
- **Agent-to-Agent Communication:** All communication happens via A2A protocol using agent IDs and remote URLs
- **Load Balancing:** Distribute workload across multiple servers for better performance

Distributed Architecture Diagram





How Distributed Communication Works

1. **Agent Discovery:** Agents register their capabilities and endpoints with the A2A registry
2. **Agent Addressing:** Each agent is addressable via `{base_url}/a2a + agent_id`
3. **Remote Communication:**
 - Orchestrator sends A2A request to: `http://server1:8001/a2a`
 - Request body includes: `{"method": "agent.query", "params": {"agent_id": "planner_agent_v1", ...}}`
 - Agent processes and returns result via A2A response
4. **Sequential Flow:** Planner → Researcher → Reasoner → Synthesizer (each via A2A protocol)

Configuration for Distributed Deployment

To deploy agents on different servers, update `config.yaml`:

```

AGENT_ENDPOINTS:
  planner_url: http://server1.example.com:8001
  researcher_url: http://server2.example.com:8002
  reasoner_url: http://server3.example.com:8003
  synthesizer_url: http://server4.example.com:8004
  
```

Local Development (default):

```

AGENT_ENDPOINTS:
  planner_url: http://localhost:8000
  researcher_url: http://localhost:8000
  reasoner_url: http://localhost:8000
  synthesizer_url: http://localhost:8000
  
```

Specialized Agent Cards

Each specialized agent has its own agent card describing its capabilities:

Agent ID	Name	Role	Capability
planner_agent_v1	Strategic Planner	Problem decomposition	Breaks queries into 3-4 actionable steps
researcher_agent_v1	Information Researcher	Knowledge gathering	Searches vector stores and extracts findings
reasoner_agent_v1	Logic & Reasoning	Logical analysis	Applies reasoning to draw conclusions
synthesizer_agent_v1	Information Synthesizer	Response generation	Combines steps into coherent final answer

Benefits of Distributed Architecture

- Resource Optimization:** Deploy resource-intensive agents (Researcher) on GPU servers, lighter agents on CPU servers
- Fault Tolerance:** If one agent fails, others continue functioning
- Independent Scaling:** Scale specific agents based on load (e.g., multiple Researcher instances)
- Technology Flexibility:** Each agent can use different LLM backends (GPT-4, Claude, Ollama, etc.)
- Security Isolation:** Sensitive operations can run in isolated environments
- Geographic Distribution:** Deploy agents closer to data sources for lower latency

3.1 Why A2A Protocol Implementation?

Enhanced Interoperability: The A2A protocol enables the agentic_rag system to communicate with other AI agents using a standardized protocol, breaking down silos between different AI systems and frameworks.

Scalable Multi-Agent Workflows: By implementing A2A, the system can participate in complex multi-agent workflows where different agents handle specialized tasks (document processing, analysis, synthesis) and collaborate to solve complex problems.

Industry Standard Compliance: A2A is an open standard developed by Google, ensuring compatibility with other A2A-compliant agents and future-proofing the system.

Enterprise-Grade Security: A2A includes built-in security mechanisms including authentication, authorization, and secure communication protocols.

Agent Discovery: The protocol enables automatic discovery of other agents and their capabilities, allowing for dynamic agent composition and task delegation.

3.2 A2A Implementation Architecture

The A2A implementation consists of several key components:

3.2.1 Core A2A Infrastructure

- A2A Models** (a2a_models.py): Pydantic models for JSON-RPC 2.0 communication
- A2A Handler** (a2a_handler.py): Main request handler and method router
- Task Manager** (task_manager.py): Long-running task execution and status tracking

- **Agent Registry** (`agent_registry.py`): Agent discovery and capability management
- **Agent Card** (`agent_card.py`): Capability advertisement and metadata

3.2.2 Supported A2A Methods

The system supports the following A2A protocol methods:

- `document.query`: Query documents using RAG with intelligent routing
- `document.upload`: Process and store documents in vector database
- `agent.query`: **NEW** - Query specialized CoT agents (Planner, Researcher, Reasoner, Synthesizer) for distributed reasoning
- `task.create`: Create long-running tasks for complex operations
- `task.status`: Check status of running tasks
- `task.cancel`: Cancel running tasks
- `agent.discover`: Discover other agents and their capabilities
- `agent.register`: Register new agents with the A2A registry
- `agent.card`: Get agent capability information
- `health.check`: System health and status check

3.3 A2A Endpoints

The system exposes the following A2A endpoints:

- `POST /a2a`: Main A2A protocol endpoint for agent communication
- `GET /agent_card`: Get the agent's capability card
- `GET /a2a/health`: A2A health check endpoint

3.4 Usage Examples

3.4.1 Basic A2A Communication

```
# Query documents via A2A protocol
curl -X POST http://localhost:8000/a2a \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "document.query",
    "params": {
      "query": "What is machine learning?",
      "collection": "PDF",
      "use_cot": true
    },
    "id": "1"
  }'
```

3.4.2 Task Management

```
# Create a long-running task
curl -X POST http://localhost:8000/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "task.create",
  "params": {
    "task_type": "document_processing",
    "params": {
      "document": "large_document.pdf",
      "chunk_count": 100
    }
  },
  "id": "2"
}'

# Check task status
curl -X POST http://localhost:8000/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "task.status",
  "params": {
    "task_id": "task-id-from-previous-response"
  },
  "id": "3"
}'
```

3.4.3 Agent Discovery

```
# Discover agents with specific capabilities
curl -X POST http://localhost:8000/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "agent.discover",
  "params": {
    "capability": "document.query"
  },
  "id": "4"
}'

# Discover specialized CoT agents
curl -X POST http://localhost:8000/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "agent.discover",
  "params": {
    "capability": "agent.query"
  }
}'
```

```
    },  
    "id": "5"  
  }'
```

Get agent card

```
curl -X GET http://localhost:8000/agent_card
```

3.4.4 Specialized Agent Query (Distributed CoT)

Query the Planner Agent

```
curl -X POST http://localhost:8000/a2a \  
-H "Content-Type: application/json" \  
-d '{  
  "jsonrpc": "2.0",  
  "method": "agent.query",  
  "params": {  
    "agent_id": "planner_agent_v1",  
    "query": "How does machine learning work?"  
  },  
  "id": "6"  
}'
```

Query the Researcher Agent

```
curl -X POST http://localhost:8000/a2a \  
-H "Content-Type: application/json" \  
-d '{  
  "jsonrpc": "2.0",  
  "method": "agent.query",  
  "params": {  
    "agent_id": "researcher_agent_v1",  
    "query": "How does machine learning work?",  
    "step": "Understand the basic concept of ML"  
  },  
  "id": "7"  
}'
```

Query the Reasoner Agent

```
curl -X POST http://localhost:8000/a2a \  
-H "Content-Type: application/json" \  
-d '{  
  "jsonrpc": "2.0",  
  "method": "agent.query",  
  "params": {  
    "agent_id": "reasoner_agent_v1",  
    "query": "How does machine learning work?",  
    "step": "Analyze the key components",  
    "context": [{"content": "Research findings about ML algorithms..."}]  
  },  
  "id": "8"  
}'
```

```
# Query the Synthesizer Agent
curl -X POST http://localhost:8000/a2a \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "agent.query",
    "params": {
      "agent_id": "synthesizer_agent_v1",
      "query": "How does machine learning work?",
      "reasoning_steps": [
        "ML is a subset of AI...",
        "It uses algorithms to learn from data...",
        "Key components include training and inference..."
      ]
    },
    "id": "9"
  }'
```

3.5 Testing A2A Implementation

The A2A implementation includes comprehensive tests covering all functionality:

3.5.1 Running Tests

```
# Run all A2A tests
python tests/run_a2a_tests.py

# Run specific test categories
python -m pytest tests/test_a2a.py::TestA2AModels -v
python -m pytest tests/test_a2a.py::TestA2AHandler -v
python -m pytest tests/test_a2a.py::TestTaskManager -v
python -m pytest tests/test_a2a.py::TestAgentRegistry -v
```

3.5.2 Test Coverage

The test suite includes:

- **Unit Tests:** Individual component testing
- **Integration Tests:** End-to-end workflow testing
- **Async Tests:** Asynchronous operation testing
- **Error Handling:** Error condition testing
- **Model Validation:** Data model testing

3.6 A2A Agent Cards

The system publishes its capabilities through agent cards. There is a main agent card for the RAG system and individual cards for each specialized CoT agent:

Main RAG Agent Card

```
{
  "agent_id": "agentic_rag_v1",
  "name": "Agentic RAG System",
  "version": "1.0.0",
  "description": "Intelligent RAG system with multi-agent reasoning",
  "capabilities": [
    {
      "name": "document.query",
      "description": "Query documents using RAG with context retrieval",
      "input_schema": { ... },
      "output_schema": { ... }
    },
    {
      "name": "agent.query",
      "description": "Query specialized CoT agents for distributed reasoning",
      "input_schema": {
        "agent_id": "planner_agent_v1 | researcher_agent_v1 | reasoner_agent_v1 | synthesizer_agent_v1",
        ...
      },
      "output_schema": { ... }
    }
  ],
  "endpoints": {
    "base_url": "http://localhost:8000",
    "authentication": { ... }
  }
}
```

Specialized Agent Cards

Each CoT agent has its own card accessible via agent discovery:

```
# Discover all specialized agents
curl -X POST http://localhost:8000/a2a \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "method": "agent.discover", "params": {"capability": "agent.query"}, "id": "1"}'
```

This returns agent cards for:

- **Planner Agent** (`planner_agent_v1`): Problem decomposition and strategic planning
- **Researcher Agent** (`researcher_agent_v1`): Information gathering from vector stores
- **Reasoner Agent** (`reasoner_agent_v1`): Logical reasoning and analysis
- **Synthesizer Agent** (`synthesizer_agent_v1`): Final answer synthesis

3.7 Using A2A Chat Interface in Gradio

The Gradio interface includes an **A2A Chat Interface** tab that allows you to interact with the distributed CoT agents:

1. **Navigate to A2A Chat Interface tab** in the Gradio UI
 2. **Enable "Use Chain of Thought"** checkbox to activate distributed CoT reasoning
 3. **Ask a question** and watch as it orchestrates through all specialized agents:
 -  Planner breaks down the query
 -  Researcher gathers information for each step
 -  Reasoner applies logical analysis
 -  Synthesizer combines into final answer
 4. **View step-by-step progress** as each agent processes via A2A protocol
- Specialized agents are automatically registered on server startup
 - All communication happens via A2A protocol (HTTP + JSON-RPC 2.0)

Example Query Flow:

```
User: "What is machine learning?"
  ↓ (A2A: agent.query → planner_agent_v1)
Planner: Creates 4 steps
  ↓ (A2A: agent.query → researcher_agent_v1) × 4
Researcher: Gathers info for each step
  ↓ (A2A: agent.query → reasoner_agent_v1) × 4
Reasoner: Analyzes each step
  ↓ (A2A: agent.query → synthesizer_agent_v1)
Synthesizer: Combines into final answer
  ↓
User: Receives comprehensive answer with sources
```

3.8 Benefits of A2A Integration

1. **Interoperability:** Seamless communication with other A2A-compliant agents
2. **Scalability:** Support for complex multi-agent workflows
3. **Standardization:** Industry-standard communication protocol
4. **Discovery:** Automatic agent and capability discovery
5. **Security:** Built-in authentication and authorization
6. **Future-Proofing:** Compatibility with evolving agent ecosystems
7. **Distributed Processing:** Each agent can run on separate infrastructure
8. **Load Balancing:** Distribute workload across multiple servers

3.9 Configuration

A2A Endpoint Configuration (for distributed deployment):

Edit `config.yaml` to specify agent endpoints:

```
AGENT_ENDPOINTS:
  planner_url: http://localhost:8000      # or remote server
```

```
researcher_url: http://localhost:8000 # or remote server
reasoner_url: http://localhost:8000 # or remote server
synthesizer_url: http://localhost:8000 # or remote server
```

Basic A2A functionality requires no additional configuration. The system automatically:

- Initializes A2A handlers on startup
- Registers main RAG agent and specialized CoT agents
- Starts task management services
- Enables agent discovery
- Loads agent endpoint URLs from config

3.10 Troubleshooting

Common Issues

1. **Async Test Failures:** Ensure `pytest-asyncio` is installed
2. **Import Errors:** Verify all A2A modules are in the Python path
3. **Task Timeouts:** Check task manager configuration for long-running tasks

Debug Commands

```
# Check A2A health
curl -X GET http://localhost:8000/a2a/health

# View agent capabilities
curl -X GET http://localhost:8000/agent_card

# Test basic functionality
python -c "from a2a_models import A2ARequest; print('A2A models working')"
```

Ownership

Inspired from oracle ai development advancement in ai in current days.